



1. @ViewChild()

Explanation :

1. Used to get a reference to a single DOM element, directive, or component that exists in the component's own template.
2. Returns the first matching element.
3. Access child component or DOM element

Lifecycle Tip:

@ViewChild is available after `ngAfterViewInit()`.

Example 



1. @ViewChild()

```
// child.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `<p>Child component works!</p>`
})
export class ChildComponent {
  sayHello() {
    console.log('Hello from ChildComponent!');
  }
}

// parent.component.ts
import { Component, ViewChild, AfterViewInit } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  template: `
    <h2>Parent Component</h2>
    <app-child></app-child>
  `
})
export class ParentComponent implements AfterViewInit {
  @ViewChild(ChildComponent) childComp!: ChildComponent;

  ngAfterViewInit() {
    this.childComp.sayHello();
  }
  //=> Access child method after view init
}
```



2. @ViewChildren

Explanation :

1. Used to get references to multiple elements/components/directives inside the view template.
2. Returns a QueryList (which behaves like an array).
3. Access multiple elements or components.

Lifecycle Tip:

@ViewChildren is also available after `ngAfterViewInit()`.

Example 

2. @ViewChildren

```
// child.component.ts
@Component({
  selector: 'app-child',
  template: `<p>Child Works!</p>`
})
export class ChildComponent {
  id = Math.floor(Math.random() * 100);
}

// parent.component.ts
import { Component, ViewChildren, QueryList, AfterViewInit } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  template: `
    <h2>Parent Component</h2>
    <app-child></app-child>
    <app-child></app-child>
    <app-child></app-child>
  `
})
export class ParentComponent implements AfterViewInit {
  @ViewChildren(ChildComponent) children!: QueryList<ChildComponent>;

  ngAfterViewInit() {
    this.children.forEach(child => console.log(child.id));
  }
}
```



3. @ContentChild()

Explanation :

1. Used to access a single projected content element that is passed from a parent using `<ng-content>`.
2. Belongs to Content DOM, not View DOM.
3. Access projected content from parent

Lifecycle Tip:

`@ContentChild` is available after `ngAfterContentInit()`.

Example 



3. @ContentChild()

```
// child.component.ts
import { Component, ContentChild, AfterContentInit, ElementRef } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <h3>Child Component</h3>
    <ng-content></ng-content>
  `
})
export class ChildComponent implements AfterContentInit {
  @ContentChild('para') paragraph!: ElementRef;

  ngAfterContentInit() {
    console.log(this.paragraph.nativeElement.textContent);
  }
}

// parent.component.html
<app-child>
  <p #para>Projected content from parent!</p>
</app-child>
```



4. @ContentChildren()

Explanation :

1. Used to get multiple projected elements passed into a component using `<ng-content>`.
2. Returns a `QueryList` like `@ViewChildren`.
3. Access multiple projected elements

Lifecycle Tip:

`@ContentChild` is available after `ngAfterContentInit()`.

Example 

4. @ContentChildren()

```
// child.component.ts
import { Component, ContentChildren, AfterContentInit, QueryList, ElementRef } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `
    <h3>Child Component</h3>
    <ng-content></ng-content>
  `
})
export class ChildComponent implements AfterContentInit {
  @ContentChildren('para') paragraphs!: QueryList<ElementRef>;

  ngAfterContentInit() {
    this.paragraphs.forEach(p => console.log(p.nativeElement.textContent));
  }
}

<!-- parent.component.html -->
<app-child>
  <p #para>Paragraph 1</p>
  <p #para>Paragraph 2</p>
</app-child>
```